

ALGO VISION

AI / DATA SCIENCE ANALYSIS REPORT

Day 01 - Day 02 Deliverables

Prepared By

Shreyam Majumdar

Domain: AI / Data Science

Visualization Requirements · Algorithm Research · Learning Experience

Table of Contents

1. Executive Summary	3
2. Platform Capability Analysis	3
3. Visualization Requirements — Sorting	4
4. Visualization Requirements — Searching	5
5. Machine Learning Visualization Analysis	6
6. Algorithm Comparison Research — Sorting	7
7. Algorithm Comparison — Searching, Graph & DP	8
8. ML Model Comparison	9
9. Learning Experience Recommendations	10
10. AI-Powered Future Features	11
11. Development Roadmap	12
12. Conclusion	13

1. Executive Summary

AlgoVision is an interactive educational platform designed to improve conceptual understanding of Data Structures, Algorithms, and Machine Learning through real-time visual simulation and experimentation. Unlike traditional resources that rely on static textbook examples, AlgoVision allows learners to observe algorithm execution live, interact with custom data inputs, compare algorithmic approaches, and analyse performance metrics dynamically.

44+	9	4	3
Algorithm Simulations	Learning Categories	ML Models	Core Analysis Areas

This report covers:

- Visualization Requirements Analysis for all algorithm and ML modules
- Algorithm Comparison Research based on time and space complexity
- Enhanced Learning Experience flow with interactive and adaptive elements
- Future AI-powered features including adaptive tutoring and personalised learning paths

2. Platform Capability Analysis

Category	Algorithms / Modules	Count	Level
Sorting	Bubble, Selection, Insertion, Merge, Quick, Heap, Counting, Radix, Shell, Bucket	10	Beginner – Advanced
Searching	Linear, Binary, Jump, Interpolation, Exponential	5	Beginner – Intermediate
Graph	BFS, DFS, Dijkstra, Topological Sort	4	Intermediate – Advanced
Tree	Inorder, Preorder, Postorder, Level Order, BST Insert	5	Intermediate
Stack & Queue	Stack Push/Pop, Queue, Valid Parentheses, Next Greater Element	4	Beginner
Linked List	Reverse, Detect Cycle, Merge Sorted, Find Middle	4	Beginner – Intermediate
Recursion	Tower of Hanoi, N-Queens, Rat in a Maze, Subset Generation	4	Intermediate – Advanced
Dynamic Prog.	Fibonacci, 0/1 Knapsack, LCS, Coin Change	4	Advanced
Machine Learning	Linear Regression, K-Means, KNN, Decision Tree	4	AI / ML Level

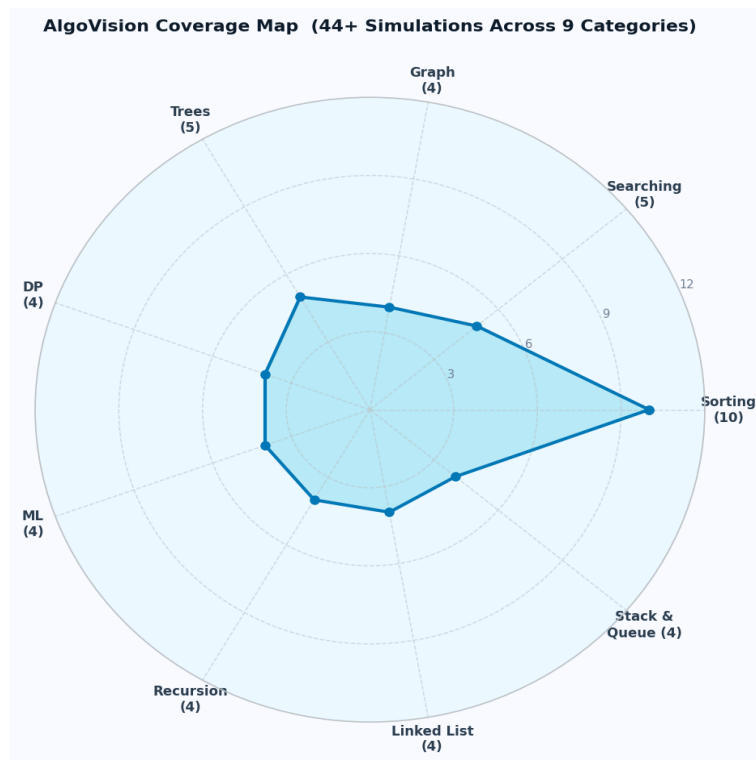


Figure 1 — AlgoVision Coverage Map (44+ Simulations)

3. Visualization Requirements — Sorting Algorithms

Each sorting algorithm has a unique execution pattern requiring tailored visual elements to convey its mechanism clearly. The table below maps every algorithm to its recommended visualization components, live metrics, and learning outcome.

Algorithm	Current Viz	Recommended Enhancement	Key Metrics	Learning Outcome
Bubble Sort	Bar + swap anim.	Live comparison counter, step mode, pause/resume	Swaps, Comparisons, Time	Adjacent comparison logic
Selection Sort	Sequential scan	Minimum-pointer tracker, unsorted-zone highlight	Min selections, Passes	Minimum-finding loop
Insertion Sort	Element shifting	Sorted/unsorted boundary, shift counter	Shifts, Comparisons	Incremental insertion
Merge Sort	Array splitting	Recursion tree overlay with depth labels	Recursive calls, Merge ops	Divide and Conquer
Quick Sort	Partition process	Dynamic pivot highlight, partition zone colours	Pivot selections, Partitions	Partition efficiency
Heap Sort	Array view	Interactive binary heap tree alongside array	Heapify ops, Extractions	Tree-based sorting
Counting Sort	Step-wise update	Frequency array construction animation	Frequency counts, Range k	Non-comparison sorting
Radix Sort	Digit grouping	Bucket distribution per digit pass	Digit passes, Bucket fills	LSD / MSD sorting
Shell Sort	Gap comparisons	Gap sequence visualised across iterations	Gap value, Comparisons/pass	Diminishing increment
Bucket Sort	Bucket assignment	Scatter distribution + intra-bucket sort highlight	Buckets used, Distribution	Distribution-based sorting

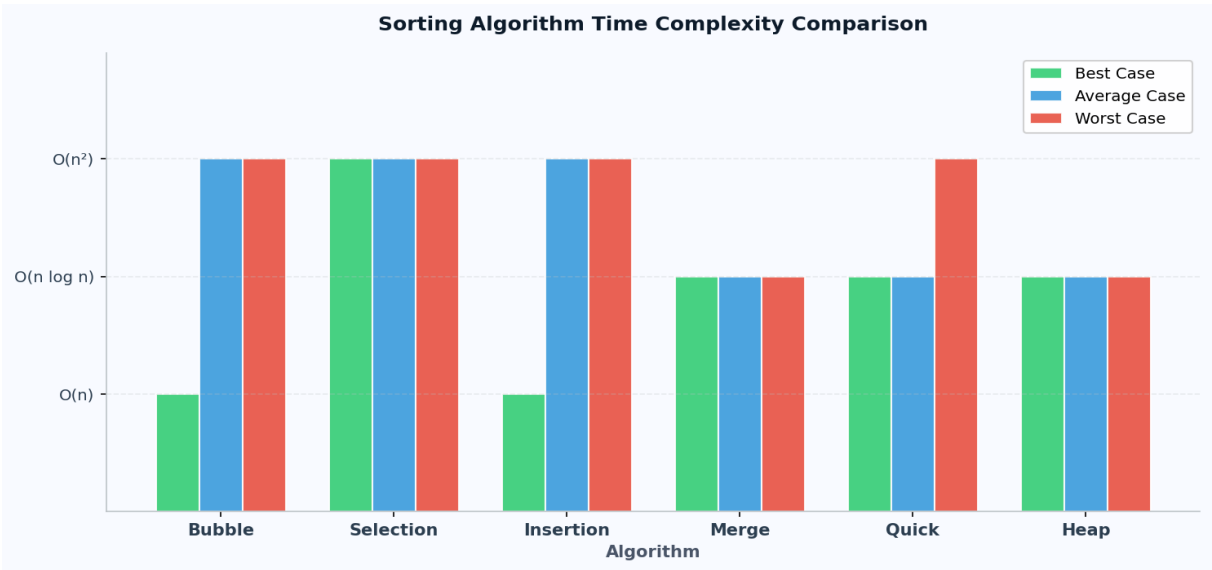


Figure 2 — Sorting Algorithm Time Complexity Comparison

4. Visualization Requirements — Searching Algorithms

AlgoVision includes five searching algorithms covering a wide complexity spectrum from $O(n)$ to $O(\log \log n)$. Each requires specific visual cues to communicate its traversal or estimation strategy to learners effectively.

Algorithm	Complexity	Visualization Elements	Display Metrics	Strength / Limitation
Linear Search	$O(n)$	Sequential index scan, visited-cell colouring	Current index, Comparisons, Match status	Simple; slow for large n
Binary Search	$O(\log n)$	Low / Mid / High pointers, shrinking range shade	Search space remaining, Iterations	Fast; requires sorted data
Jump Search	$O(\sqrt{n})$	Block-jump indicators, linear scan zone highlight	Block size, Jumps, Scan comparisons	Hybrid; no full-sort needed
Interpolation Search	$O(\log \log n)$	Estimated probe position, actual vs estimated gap	Probe index, Estimation error	Fastest for uniform data
Exponential Search	$O(\log n)$	Range-doubling animation, binary search sub-zone	Doubling steps, Binary iterations	Efficient on unbounded arrays

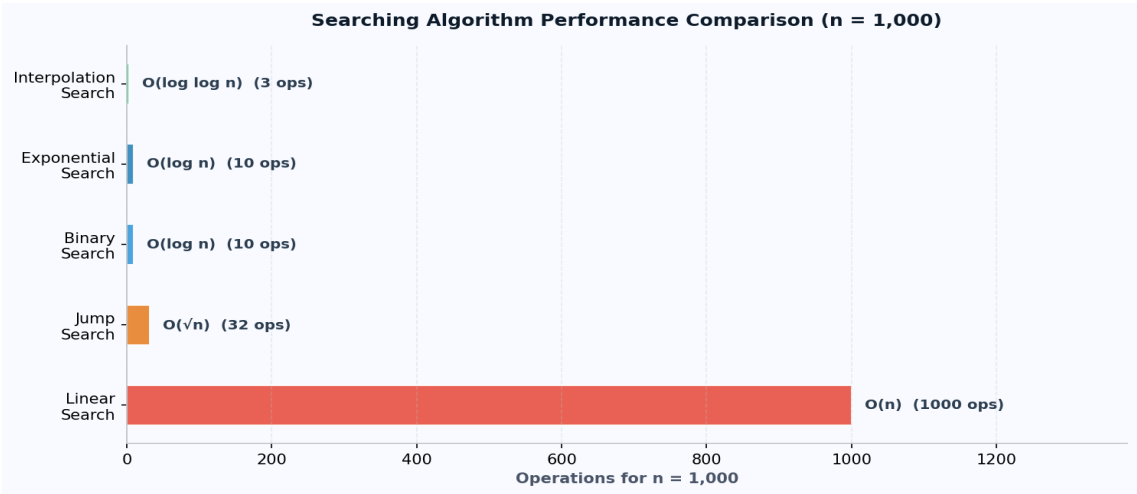


Figure 3 — Searching Algorithm Performance Comparison (n = 1,000)

Recommended Live Dashboard Metrics

Metric	Description
Current Index	Pointer showing the element currently under examination
Remaining Search Space	Count of array elements still in scope for the search
Comparisons Performed	Running total of comparisons since the search started
Execution Time (ms)	Wall-clock time to find the target or exhaust the search space

5. Machine Learning Visualization Analysis

The Machine Learning module is the most analytically rich section of AlgoVision. Four models — Linear Regression, K-Means, KNN, and Decision Tree — span supervised, unsupervised, and classification paradigms. Each requires distinct interactive visualizations and live metric panels to build genuine intuition.

Model	Type	Required Visualizations	Interactive Controls	Key Metrics
Linear Regression	Supervised Prediction	Scatter plot, best-fit line animation, loss curve, gradient descent step indicator	Learning rate, Epochs, Dataset size, Noise level	MSE, R-squared, Convergence rate
K-Means	Unsupervised Clustering	Cluster formation per iteration, centroid movement trail, cluster boundary overlay	K value, Max iterations, Dataset size	SSE, Silhouette score, Iterations to converge
KNN	Supervised Classification	Decision region shading, K-nearest neighbour highlight, class-voting bar	K value, Distance metric (Euclidean / Manhattan)	Accuracy, Precision, Recall, F1-Score
Decision Tree	Supervised Classification	Tree growth animation, node-split explanation, colour-coded decision paths	Max depth, Min samples per split, Feature selector	Gini Index, Information Gain, Accuracy

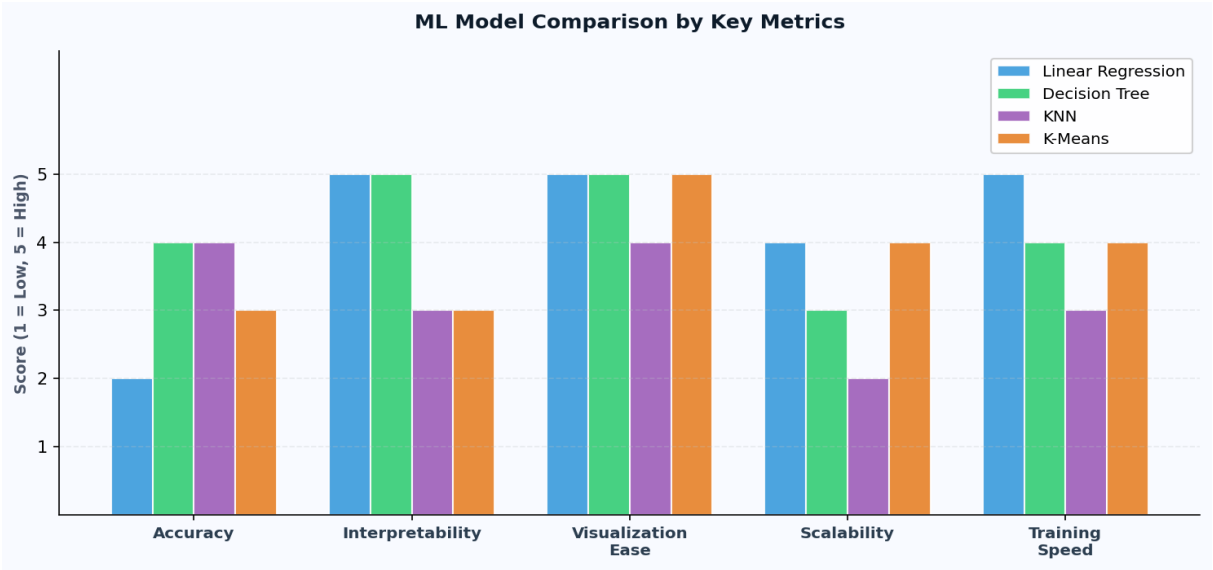


Figure 4 — ML Model Comparison by Key Metrics (Score 1 = Low, 5 = High)

6. Algorithm Comparison Research — Sorting

A full time and space complexity breakdown for all ten sorting algorithms in AlgoVision. This serves as the research foundation for the proposed Algorithm Race Mode.

Algorithm	Best	Average	Worst	Space	Stable ?	Best Use Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Educational; small data
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Minimising swaps
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Nearly-sorted arrays
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Linked lists, large data
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	General purpose (fastest avg)
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Memory-constrained systems
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Yes	Integer data, small range
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes	Fixed-length integer keys
Shell Sort	$O(n \log n)$	$O(n \log^2 n)$	$O(n^2)$	$O(1)$	No	Medium-sized datasets
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	Yes	Uniform float distribution

Algorithm Race Mode — Priority Comparison Pairs

Comparison Pair	Purpose	Key Insight for Learner
Bubble Sort vs Quick Sort	Dramatic efficiency gap	$O(n^2)$ vs $O(n \log n)$ visible in real execution
Merge Sort vs Quick Sort	Stable vs unstable D&C;	Memory overhead vs average-case speed trade-off
Insertion Sort vs Shell Sort	Gap-sequence benefit	How diminishing increments improve insertion
Counting Sort vs Merge Sort	Non-comparison vs comparison	Domain-specific algorithm advantages

7. Algorithm Comparison — Searching, Graph & Dynamic Programming

Searching Algorithm Comparison

Algorithm	Complexity	Best	Worst	Prerequisite	Ideal Dataset
Linear Search	$O(n)$	$O(1)$	$O(n)$	None	Unsorted / small
Binary Search	$O(\log n)$	$O(1)$	$O(\log n)$	Sorted array	Large sorted arrays
Jump Search	$O(\sqrt{n})$	$O(1)$	$O(\sqrt{n})$	Sorted array	Medium sorted arrays
Interpolation Search	$O(\log \log n)$	$O(1)$	$O(n)$	Sorted uniform	Uniform distribution
Exponential Search	$O(\log n)$	$O(1)$	$O(\log n)$	Sorted array	Unbounded / infinite

Graph Algorithm Comparison

Algorithm	Strategy	Time	Space	Best For
BFS	Level-by-level traversal	$O(V+E)$	$O(V)$	Shortest path (unweighted)
DFS	Depth-first recursion / stack	$O(V+E)$	$O(V)$	Cycle detection, topological order
Dijkstra	Greedy shortest-path (priority Q)	$O(V^2)$	$O(V)$	Weighted shortest path
Topological Sort	DFS-based linear ordering	$O(V+E)$	$O(V)$	Dependency resolution, DAGs

Dynamic Programming Problem Comparison

Problem	Time	Space	DP Type	Visualization Approach
Fibonacci	$O(n)$	$O(n)$	Bottom-Up memoization	Table fill animation, recursion tree
0/1 Knapsack	$O(n \times W)$	$O(n \times W)$	Bottom-Up 2D table	2D DP grid construction, cell trace
LCS	$O(m \times n)$	$O(m \times n)$	Bottom-Up 2D table	Grid build + backtrack path highlight
Coin Change	$O(n \times A)$	$O(A)$	Bottom-Up 1D array	1D DP array evolution per coin

8. ML Model Comparison

Model	Type	Use Cases	Advantages	Limitations	Visualization
Linear Regression	Supervised Prediction	Price prediction, trend forecasting, risk modelling	Simple, interpretable, very fast to train	Only captures linear relationships	Scatter + regression line + loss curve
K-Means	Unsupervised Clustering	Customer segmentation, image compression, anomaly detection	Scalable, easy to implement and understand	Fixed K required, sensitive to outliers	Cluster scatter + centroid movement
KNN	Supervised Classification	Recommendation engines, pattern recognition, medical diagnosis	No training phase, highly intuitive	Slow for large n, high memory usage	Decision boundary + neighbour highlight
Decision Tree	Supervised Classification	Fraud detection, medical triage, rule extraction	Highly interpretable, handles mixed features	Prone to overfitting without pruning	Tree growth + node-split animation

ML Experiment Sandbox — Tunable Parameters

Allowing learners to adjust hyperparameters and observe the immediate effect builds stronger ML intuition than any static diagram.

Model	Tunable Parameters	Observable Effect on Visualization
Linear Regression	Learning rate, Epochs, Dataset noise, Dataset size	Loss curve shape, convergence speed, regression line stability
K-Means	K value, Max iterations, Dataset size, Random seed	Cluster formation quality, SSE reduction curve, convergence count
KNN	K neighbours, Distance metric (Euclidean / Manhattan)	Decision boundary smoothness, accuracy vs K trade-off curve
Decision Tree	Max depth, Min samples per split, Feature selection	Tree complexity growth, leaf purity, overfitting vs underfitting

9. Learning Experience Recommendations

The current AlgoVision flow is minimal: select an algorithm, run its visualization. This passive experience limits retention and engagement. The proposed eight-step flow introduces structured learning paths, theory-first scaffolding, comparison modes, quizzes, and progress tracking to create a complete active-learning cycle.

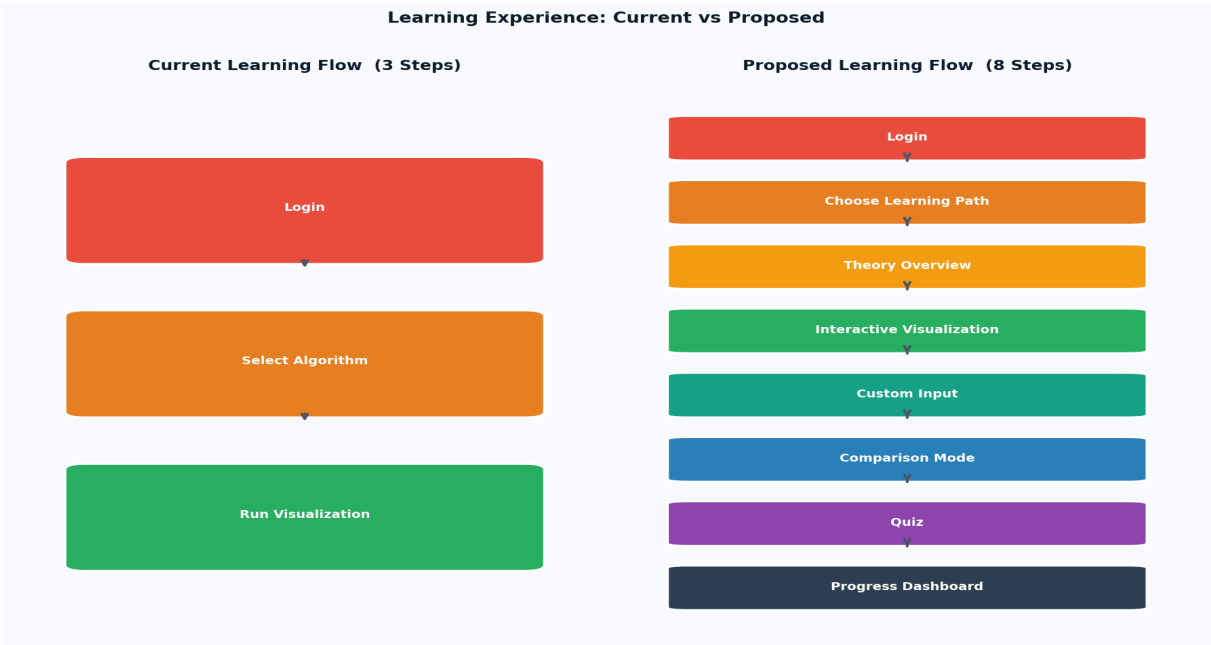


Figure 5 — Learning Experience: Current (3-Step) vs Proposed (8-Step) Flow

Enhancement Recommendations

#	Feature	Description	Priorty
1	Custom Dataset Input	Users enter their own arrays (e.g. 10 4 8 3 9) instead of random-only data, enabling targeted experimentation	High
2	Algorithm Race Mode	Side-by-side real-time comparison of two algorithms on the same dataset with live operation counters	Very High
3	Complexity Visualizer	Live display of time and space complexity, actual operation counts, and memory usage as the algorithm runs	High
4	Theory Cards	Brief theory overview before each algorithm begins — covers definition, use cases, and complexity	Medium
5	Integrated Quiz	Short 3–5 question quiz after each algorithm on complexity, edge cases, and practical trade-offs	High
6	Progress Dashboard	Tracks topics completed, quiz scores, practice time, accuracy trend, and daily learning streak	High
7	ML Sandbox	Real-time hyperparameter tuning with instant visual feedback for all four ML models	Very High

10. AI-Powered Future Features

The following AI-powered enhancements would elevate AlgoVision from a visualizer to a fully intelligent, adaptive learning platform. They are recommended for phased implementation after core visualization improvements are complete.

Feature	Description	Technology Stack	Priority
AI Tutor	Step-by-step natural-language explanations for every algorithm decision. "Why was this swap performed?", "Why was this pivot chosen?", "Why did the centroid move?"	LLM integration, step-annotation engine, WebSocket streaming	Very High
Personalised Learning Paths	After completing Bubble Sort, the system recommends Merge Sort or Quick Sort based on quiz scores and observed conceptual gaps rather than a fixed curriculum	Collaborative filtering, knowledge-graph traversal	High
Adaptive Difficulty	Automatically adjusts quiz difficulty, dataset complexity, and visualization speed based on the learner's rolling performance history across sessions	Reinforcement learning, performance modelling	High
Performance Analytics	Visual dashboard predicting weak areas, strong areas, and suggested revision topics with learning velocity trends over time	Time-series analysis, predictive modelling	Medium
Voice-Based Tutor	Spoken explanations synchronised to each visualization step, enabling fully accessible and hands-free learning mode	Text-to-speech, real-time sync with animation engine	Medium
Interview Prep Mode	Curated timed problem sets mapped to algorithm categories with expected complexity answers aligned to technical interview standards	Problem bank, evaluation engine, spaced repetition	Medium

11. Development Roadmap

Phase	Timeline	Focus Area	Key Deliverables	Impact
Phase 1	Week 1–2	Visualization Depth	Enhanced metrics, step/pause mode, recursion tree for Merge Sort and Quick Sort	★★★★★ ★
Phase 2	Week 3–4	Comparison Mode	Algorithm Race Mode side-by-side, Complexity Visualizer panel	★★★★★ ★
Phase 3	Week 5–6	ML Enhancements	ML Sandbox, KNN decision regions, loss curve, hyperparameter controls	★★★★★ ★
Phase 4	Week 7–8	Learning Analytics	Quiz system, Progress Dashboard, topic-completion tracking	★★★★★
Phase 5	Week 9–10	AI Tutor (MVP)	Step-explanation engine, AI-generated hints per algorithm step via LLM	★★★★★ ★
Phase 6	Week 11–12	Personalization	Adaptive recommendations, personalised learning paths, performance prediction	★★★★★

12. Conclusion

AlgoVision already provides a strong foundation with 44+ algorithm and ML simulations across 9 categories — comprehensive coverage of every major concept a Computer Science student would encounter. The platform's current strength is its breadth.

This analysis reveals that the highest-impact opportunity is not adding more algorithms but dramatically deepening the learning experience for those that already exist. The three most critical improvements are:

1. Visualization Depth

Adding recursion trees, live metrics, comparison counters, and step-by-step mode transforms passive observation into genuine conceptual understanding. This alone would dramatically increase the educational value of every existing module.

2. Algorithm Comparison Mode

Running two algorithms simultaneously on the same dataset provides the clearest possible demonstration of why time complexity matters in practice. No textbook explanation matches the impact of watching Bubble Sort and Quick Sort race on identical data.

3. ML Experiment Sandbox

Allowing users to tune hyperparameters (learning rate, K, tree depth) and immediately see the effect builds ML intuition that no static diagram or lecture can match. This is particularly powerful for showing the bias-variance trade-off in Decision Trees and the effect of K on KNN decision boundaries.

With the proposed phased enhancements, AlgoVision has the potential to evolve into a fully intelligent, adaptive learning platform — combining interactive simulation, data-driven personalisation, and AI-assisted tutoring into a genuinely differentiated resource for algorithm education.